Secret Bitstream (Base64): cli2EARMSajA

Fully-supervised

RNN-Stega: Richards are also in the film and the film shows us the most disgusting and violent aspects of the movie.The plot revolves around a group of people who are attacked and murdered.

VAE-Stega: listen. the music and the music, the music, music of all the actors, the music, and the music! this movie is not worth your eyes.

ADG: Richian is condemned to discover the facts that blackmails people through them.

Training-free

NLS: There was so much I wanted to see in this film. I was hoping it was a little more like what I was hoping for from the original film and not what I do not expect.

SAAC: I was looking forward to going as Keanu. I really wanted to do a little red carpet before I die.

Ours: I was genuinely underwhelmed by the film, and I'm afraid the only thing that stood out to me was the final scene. I had been eagerly anticipating it, but unfortunately the rest of the movie fell flat for me.

Figure 4: Examples of stegotext generated by various methods about movie reviews with a similar BPW ≈ 2.5, except ADG and SAAC. Compared with other baseline methods, our method generates more reasonable sentences.

Methods	Syntactic		Semantic
	TS-BiRNN	R-BiLSTM-C	BERT-C
Fully-supervised			
RNN-Stega	94.02	93.88	96.50
VAE-Stega	94.75	95.65	96.17
Training-free			
NLS	84.60	86.21	92.25
ours	**80.29**	**84.34**	**89.61**

Table 3: Steganalysis with BPW ≈ 2.5. Each value is the classification accuracy by steganalyzer. Lower accuracy $\downarrow$ indicates better statistical imperceptibility.

et al., 2019b), R-BiLSTM-C (Niu et al., 2019) and BERT-C, to assess the performance of stego-methods in terms of syntactic similarity and semantic similarity. Among these methods, the BERT-C method, inspired by Wen et al. (2022), utilizes a BERT encoder (Devlin et al., 2019) to encode the input sequence and uses a linear classifier to determine whether the given sequence is a stegotext.

We exclude ADG and SAAC from our analysis since their BPW settings are difficult to adjust, leading to an unfair comparison. Moreover, due to their high BPW settings (BPW > 5), ADG and SAAC cause significant Psic Effect, which makes it difficult to classify their stegotext accurately through syntactic analysis. Even with extremely low text quality, some of these methods can achieve almost 50% detection accuracy, which indicates a perfect but unreasonable statistical imperceptibility.

In conclusion, the results show that our method outperforms most of the previous methods.

4.5 Language Evaluation

While statistical imperceptibility can measure the similarity between text distributions, human perception still plays a significant role in assessing the **perceptual imperceptibility** of the stegotext.

To simulate the human evaluation and ensure reproducibility, we conduct language evaluation on the generated stegotext of each method using large language models. First, we attempt to collect the stegotext generated with similar BPW ≈ 2.5 using the IMDB dataset[3]. We then randomly select two sentences from each set of stegotext to compare, denoted by $<\texttt{sent}_A, \texttt{sent}_B>$. Next, GPT-3.5 is used to determine which sentence in the pair is considered high-quality. To be more specific, we compare our method with each baseline and collect ratios of how much of our stegotext is deemed to be better than the others. We sample 10K sentence pairs between our method and each baseline. Appendix A.5 illustrates the details of the implementation of language evaluation.

Table 4 demonstrates three kinds of quality measurement conducted in our experiments:

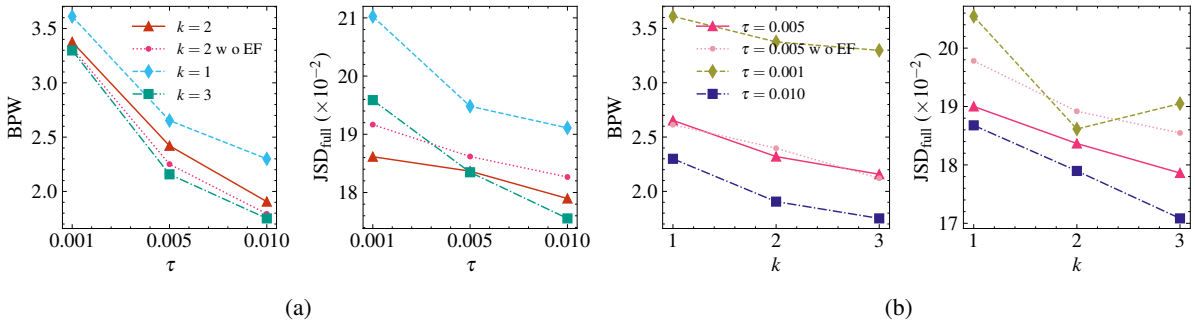
(a) **Soundness** measures the extent to which one sentence is more *logical* and *meaningful*.

(b) **Relevance** measures the extent to which one sentence is more *relevant* to the specific topic.

(c) **Engagingness** measures the extent to which one sentence is more *appealing* and *engaging*.

Our three measurements are all close to 1.0 when compared to the ground truth covertext, which indicates a high level of similarity. Most supervised

[3]For ADG and SAAC, the BPW is set to 4.931 and 4.471.



(a) (b)

Figure 5: Curves of BPW and $\mathrm{JSD_{full}}$ with respect to: (a) different threshold τ (b) different context size k

Eval.	GT	Fully-supervised			Training-free	
		RNN	VAE	ADG	NLS	SAAC
Sound.	0.788	3.812	8.363	8.042	1.373	1.654
Relev.	1.196	2.397	3.608	5.345	2.479	3.850
Engag.	1.157	5.386	9.267	7.224	1.924	2.380
Avg.	**1.047**	**3.865**	**7.080**	**6.870**	**1.926**	**2.628**

Table 4: Language evaluation results. Each value represents the ratio of how much our stegotext is considered better than the particular method. For example, a ratio of 3.812 means that the number of our stegotext considered better is 3.812× more than the other method.

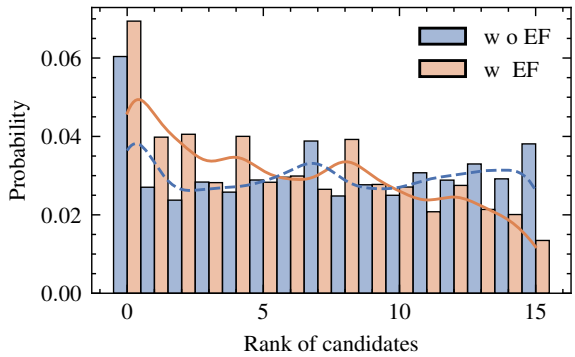


Figure 6: Distribution of the rank of selected candidates.

methods aim to approximate covertext distribution, which results in a large gap between readability and imperceptibility. Training-free methods, however, are more effective at balancing both perceptual and statistical imperceptibility. Furthermore, our method outperforms all other baselines with an average improvement of at least 1.926×.

4.6 Ablation Study

We analyze EF coding in terms of the probability of selecting a candidate from a pool of candidates. We replace every occurrence of the candidate with its rank of likelihood in the descending-sorted candidate pool in Fig. 6. It appears that words with lower ranks are more likely to be selected, which is generally of high word probability. The shift of word distribution results in slight improvements in BPW and JSDs, as shown in Fig. 5.

Figure 5 also shows the performance of our method under different hyper-parameter settings. We can observe from the results that an increase in threshold τ and context size k would result in a decrease in BPW and JSD. According to the results, a higher sentence quality would lead to a lower embedding rate but better statistical imperceptibility. This indicates a tradeoff between embedding rate and statistical imperceptibility exists.

5 Related Work

Linguistic steganography can be divided into three types (Fridrich, 2009), selection-based (Sajedi and Jamzad, 2008; Chen and Chen, 2019), edit-based (Chang and Clark, 2014) and generation-based steganography (Kang et al., 2020; Xiang et al., 2020). However, the low embedding rate of these methods hinders their practical application, giving rise to generation-based steganography.

To encode secret messages into textual data, early generative steganography utilized the Markov model to obtain conditional probability distributions (Moraldo, 2014) for text generation. Subsequently, the development of NLP has significantly improved distribution modeling. It was first proposed by Fang et al. (2017) to use a supervised recurrent neural network (RNN) to learn the statistical language model of natural texts with a fixed-length coding dictionary. Subsequently, Yang et al. (2019a) proposed a generation-based steganography based on RNN and Huffman coding. Yang *et al.* also proposed that different neural models make a difference, such as the generative adversarial network (Yang et al., 2020) and variational autoencoder (Yang et al., 2021). Zhang et al. (2021) and

Ding et al. (2023) further presented provably secure algorithms for better statistical imperceptibility.

While supervised methods attempt to fit models to the source distribution, the restriction of the source distribution leads to a high risk of generating similar but unintelligible text due to the complicated process of embedding secret messages. To address this limitation, models pre-trained on large-scale datasets are used. Ziegler et al. (2019) proposed that arithmetic coding be used with a pre-trained GPT-2 model. Dai and Cai (2019) further proposed an improved Huffman encoding algorithm, which is intended to adjust embedding rates dynamically. Shen et al. (2020) improved the previous works with self-adjusting arithmetic coding for near-imperceptible text. Nevertheless, all of these training-free approaches employ a similar paradigm for generating the context's *next sentence*, making them impractical in actual usage.

6 Conclusion

In this work, we present a zero-shot approach based on in-context learning for linguistic steganography. For the LLM to generate a similar sentence, samples of covertext serve as the context for the LLM in the QA. We also integrate several techniques to improve the text generation process for better readability and imperceptibility of the stegotext. Our experimental results exhibit the high imperceptibility of our approach compared to other methods. In addition, the stegotext is more intelligible and resembles the covertext in many aspects. We hope this work will be a step towards the more practical applications of linguistic steganography.

7 Limitations

Our approach has indeed demonstrated advantages in producing intelligible and innocent stegotext, yet we rely heavily on the generalization capabilities of LLMs. The computation overhead of LLMs is approximately $3\times$ to $5\times$ in comparison with prior works. There may be an additional vulnerability if real-time communication is required.

Further, we have not tested the proposed method in a relatively large embedding rate setting (e.g., BPW > 5). While we believe that stegotext generated under settings like these would be highly unintelligible and impractical in real life, the limits of our method have yet to be determined.

Data integrity is another essential requirement of our steganographic method. The incompleteness of the stegotext may result in invertible damage to the covert bitstream as in most previous works. Fortunately, textual data are less likely to suffer from lossy compression issues than other types of media (e.g., image, audio, and video).

8 Ethical Considerations

The use of pre-trained LLMs may exhibit potential issues, such as insults, political biases, or gender discrimination. It should also be noted that the data underlying our method may introduce additional ethical concerns. While we did not observe such problems during our experiments, users should be aware of these issues in practice.

Acknowledgement

This research is supported by the National Key R&D Program of China under grant (No.2022YFB2703001). Thanks to all the reviewers for their insightful suggestions.

References

Mihir Bellare, Anand Desai, Eron Jokipii, and Phillip Rogaway. 1997. A concrete security treatment of symmetric encryption. In *Proc. 38th Annual Symposium on Foundations of Comput. Sci.*, pages 394–403. IEEE.

Christian Cachin. 1998. An information-theoretic model for steganography. In *Information Hiding*, pages 306–318, Berlin, Heidelberg. Springer Berlin Heidelberg.

Ching-Yun Chang and Stephen Clark. 2014. Practical linguistic steganography using contextual synonym substitution and a novel vertex coding method. *Comput. Linguistics*, 40(2):403–448.

Mark Chapman and George Davida. 1997. Hiding the hidden: A software system for concealing ciphertext as innocuous text. In *Inf. and Commun. Secur.*, pages 335–345, Berlin, Heidelberg. Springer Berlin Heidelberg.

Xianyi Chen and Sheng Chen. 2019. Text coverless information hiding based on compound and selection of words. *Soft Comput.*, 23.

Falcon Dai and Zheng Cai. 2019. Towards near-imperceptible steganographic text. In *Proc. 57th Annu. Meeting Assoc. Comput. Linguistics*, pages 4303–4308, Florence, Italy. Assoc. Comput. Linguistics.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proc. 2019 Conf. North American Assoc.*